

Implémentation d'un protocole ZKP pour l'authentification sans mot de passe

Protocole de Schnorr – Université de Yaoundé I

Fotsing Kengne Diane Iris (17T2631)

Kougoum Fotsing Pavel (21T2887)

Département d'Informatique – Université de Yaoundé I

Année Académique 2025–2026

Plan de la présentation

- 1 Problématique et contexte
- 2 Fondements théoriques
- 3 Implémentation
- 4 Résultats et discussion
- 5 Conclusion

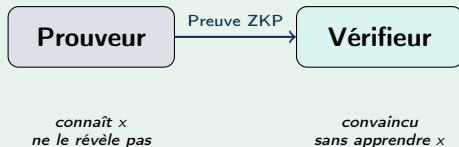
Vulnérabilités classiques

- **Transmission réseau** : interception possible
- **Rejeu** : capture et retransmission
- **Stockage serveur** : BDD souvent compromises
- **Hameçonnage** : révélation involontaire

Question centrale

Comment prouver la connaissance d'un secret **sans jamais le révéler** sur le réseau ?

Solution : Zero-Knowledge Proof



Objectifs

- ① Implémenter le protocole de **Schnorr**
- ② Échange **interactif** (3 messages)
- ③ Échange **non interactif** (Fiat-Shamir)
- ④ Évaluer la résistance aux **attaques par replay**
- ⑤ Mesurer les **performances**

Architecture technique

- Langage : **Python 3.11**
- Communication : **Sockets TCP**
- Interface : **Tkinter** (3 onglets)
- Crypto : **pycryptodome**
- 3 serveurs sur ports 5000 / 6000 / 7000

Définition (Goldwasser, Micali, Rackoff, 1985)

Un protocole ZKP entre un prouveur P et un vérifieur V satisfait :

1. Complétude

Si P connaît le secret, V accepte avec proba ≈ 1

2. Solidité

Si P triche, V rejette avec proba ≈ 1

3. Connaissance nulle

V n'apprend rien d'autre que la validité

Fondement : logarithme discret

Soit $g^q \equiv 1 \pmod{p}$. Clé privée x , clé publique $y = g^x \pmod{p}$.

Retrouver x à partir de y est computationnellement impossible pour de bons paramètres (p de 2048 bits).

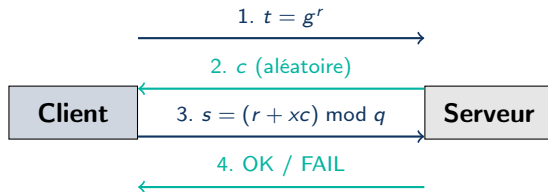
Le protocole de Schnorr (1989)

Paramètres publics : (p, q, g, y) , avec
 $y = g^x \bmod p$

1. **Engagement** : $r \xleftarrow{R} [1, q-1]$, envoie
 $t = g^r \bmod p$
2. **Défi** : le vérifieur envoie c aléatoire
3. **Réponse** : envoie $s = (r + x \cdot c) \bmod q$
4. **Vérification** : accepte si $g^s \equiv t \cdot y^c \pmod{p}$

Correctness

$$t \cdot y^c = g^r \cdot (g^x)^c = g^{r+xc} = g^s \quad \checkmark$$



Paramètres de test

p	q	g
23	11	2

Principe

Remplacer le défi interactif c par un hash :

$$c = H(t, y, \text{timestamp})$$

Le prouveur envoie (t, s, ts) en un seul message.

Calcul du défi (SHA-256)

```
def hash_to_c(t, y, timestamp):  
    donnees = f"{t}{y}{timestamp}"  
    h = hashlib.sha256(  
        donnees.encode()  
    ).hexdigest()  
    return int(h, 16) % Q
```

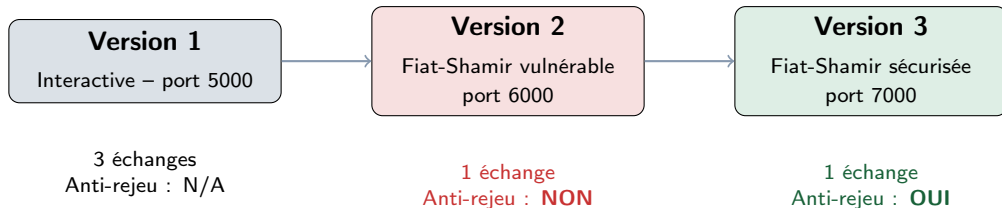
Avantage : latence divisée par 2

Version	Latence
Interactive	$T + 2 \times RTT$
Fiat-Shamir	$T + RTT$

Vérification anti-rejeu

```
if timestamp in timestamps_vus:  
    return "REJET - Rejeu"  
if abs(time.time()-ts) > 60:  
    return "REJET - Expiré"  
timestamps_vus.add(timestamp)
```

Architecture des trois versions



Interface Tkinter

- 3 onglets (une version par onglet)
- Démarrage / arrêt du serveur
- Génération des clés cryptographiques
- Capture et rejeu d'attaque

Structure des fichiers

- `params.py` – paramètres (p, q, g)
- `crypto.py` / `crypto_fs.py`
- `serveur_auth.py` / `client_auth.py`
- `pirate_rejeu.py` / `mesures_perf.py`
- `interface.py` / `serveur_secure.py`
- `serveur_secure.py`

Attaque par rejeu – démonstration

Attaque réussie – Version 2 (vulnérable)

Valeurs capturées :

$t = 16$, $s = 8$, $ts = 1744221123$

Message rejoué tel quel au serveur

Résultat : OK $\Rightarrow$ attaque réussie !

*Le serveur ne distingue pas
un message légitime d'un rejeu.*

Attaque bloquée – Version 3 (sécurisée)

Mêmes valeurs rejouées

Double vérification serveur :

❶ Timestamp déjà vu ? $\Rightarrow$ **REJET**

❷ Timestamp > 60 s ? $\Rightarrow$ **REJET**

Résultat : REJET – Rejeu ✓

Interception passive (toutes versions)

Seules les valeurs t , c et s circulent sur le réseau. Le secret x **n'apparaît jamais**, conformément à la propriété de connaissance nulle.

Performances mesurées (1 000 échantillons)

Opération	Côté	Temps (ms)
Génération des clés	Client	0.0023
Engagement ($t = g^r$)	Client	0.0018
Hash SHA-256	Client	0.0150
Réponse ($s = r + xc$)	Client	0.0004
Vérification	Serveur	0.0025
Total client		0.0195
Total serveur		0.0025

Observation clé

Charge crypto < 0,02 ms.

Goulot d'étranglement = latence réseau.

Cela justifie le passage à Fiat-Shamir.

Comparaison des méthodes

Méthode	Sécu.	Perf.
Mots de passe	Faible	Très él.
ZKP Schnorr	Élevée	Élevée
WebAuthn	Très él.	Élevée
Biométrie	Moyenne	Élevée

Réduction latence

Version	Latence
Interactive	$T + 2 \times RTT$
Fiat-Shamir	$T + RTT$

Limites identifiées

- **Post-quantique** : le logarithme discret est vulnérable aux ordinateurs quantiques
- **Horloge synchronisée** requise pour la validation des timestamps
- **Scalabilité** : stockage des timestamps problématique à grande échelle
- **Canaux auxiliaires** : timing, cache, consommation électrique

Perspectives M2

- **Intégration WebAuthn** : standard W3C sans mot de passe
- **Constant-time** : résistance aux canaux auxiliaires
- **Double Ratchet** : forward secrecy (protocole Signal)
- **Post-quantique** : cryptographie sur réseaux euclidiens

Ce que nous avons réalisé

- ✓ Implémentation complète du protocole de Schnorr en Python
- ✓ Trois versions fonctionnelles : interactive, Fiat-Shamir, sécurisée
- ✓ Démonstration d'attaque par rejeu et de sa contre-mesure efficace
- ✓ Interface graphique Tkinter unifiée et ergonomique
- ✓ Mesures de performance sur 1 000 échantillons

Message clé

S'authentifier sans jamais révéler son secret :
le protocole de Schnorr rend cela **possible, pratique et mesurable.**

Code source : <https://github.com/PavelFotsing/projet-zkp-schnorr>

Merci pour votre attention

Questions ?

Fotsing Kengne Diane Iris
17T2631

Kougoum Fotsing Pavel
21T2887

Département d'Informatique – Université de Yaoundé I – 2025/2026